

Vibe code app 8 (BAS Lite)

Generated by scripts/build_docs_pdf.py

April 15, 2026

- 1 Boss Pi Bas Lite Docker
- 2 Boss Pi — BAS Lite (Docker + easy-aso + Caddy)
 - 2.1 1. Install Docker Engine on Raspberry Pi OS
 - 2.1.1 1.1 Update the system
 - 2.1.2 1.2 Install Docker on Raspberry Pi OS
 - 2.1.3 1.3 Install Compose support
 - 2.1.4 1.4 Quick test
 - 2.2 2. BACnet JSON-RPC (diy-bacnet-server)
 - 2.3 3. Copy this folder to the Pi
 - 2.3.1 3.1 Windows / PowerShell (read this if you use Windows)
 - 2.3.2 3.2 Scripts quick reference (Windows vs Pi)
 - 2.4 4. Build and start BAS Lite
 - 2.4.1 4.0a Bootstrap script (after sync-bas-lite-to-bosspi.ps1 -SyncOnly)
 - 2.4.2 4.0 How you reach the UI (loopback vs LAN)
 - 2.4.3 4.1 FastAPI OpenAPI (operator debugging)
 - 2.5 5. SD card wear — logs and Docker writes
 - 2.5.1 5.1 Compose logging (already set)
 - 2.5.2 5.2 Prune periodically
 - 2.5.3 5.3 SQLite data volume
 - 2.6 6. Security model (OT LAN)
 - 2.7 7. Related easy-aso documentation
 - 2.8 8. Developer loop on a PC (optional)
 - 2.9 9. Runtime model
 - 2.9.1 Optional outside-air share (easy-aso-oat, profile oat)
 - 2.9.2 Multi-agent EasyASO sidecars (profile agents)
 - 2.10 10. BACnet bench workflow on Boss Pi

- 2.11 11. Troubleshooting
- 2.12 12. Stack tradeoffs
- 3 Easy Aso Upstream Multi Agent Draft
- 4 Multi-agent runtime and RPC-docked agents — draft for **easy-aso** repository
 - 4.1 1. Goals
 - 4.2 2. Conceptual model (VOLTTRON analogy)
 - 4.3 3. Proposed package layout (inside **easy-aso**)
 - 4.4 4. Public API (minimal v1)
 - 4.4.1 4.1 RpcDockedEasyASO
 - 4.4.2 4.2 Runner
 - 4.4.3 4.3 CLI (entry point)
 - 4.5 5. Configuration contract (env + optional manifest)
 - 4.5.1 5.1 Environment variables (canonical names)
 - 4.5.2 5.2 Agent manifest (JSON or YAML) — P1 but specify now
 - 4.6 6. Logging and observability (P1)
 - 4.7 7. Version compatibility
 - 4.8 8. What demo repos keep after this lands (config-only ideal)
 - 4.9 9. Migration checklist (from current BAS Lite demo)
 - 4.10 10. Phased delivery
 - 4.11 11. Test matrix (upstream)
 - 4.12 12. Appendix — Issue titles for **easy-aso** tracker
- 5 Bas Lite App8 Tutorial
- 6 BAS Lite App 8 Tutorial
 - 6.1 BACnet discovery in this stack
 - 6.2 Project positioning
- 7 Replicate Any Building Lan Topology
- 8 Recreating BAS Lite (App 8) on any HVAC OT LAN
 - 8.1 1. What must stay aligned
 - 8.2 2. Device and point identities
 - 8.3 3. Site model (no Python edits for a new building)
 - 8.4 4. Occupancy schedule + optional BACnet sidecars
 - 8.5 5. Deploy scripts from a Windows workstation
 - 8.6 6. Checklist before go-live
 - 8.7 7. Non-BACnet drivers

1 Boss Pi Bas Lite Docker

2 Boss Pi — BAS Lite (Docker + easy-aso + Caddy)

This tutorial is for **beginners** who want the **React “BAS Lite” operator UI** on a Raspberry Pi (**Pi 5 or older 3B+**) using:

- **Docker Compose** (no systemd units for the app — only Docker’s restart policy).
- **easy-aso** on PyPI (**0.1.7** in this stack): asyncio **supervisor** + **FastAPI** (/api/v1/*) and legacy JSON for the SPA (/app8/api/*), plus **optional multi-agent** sidecars (§9).
- **diy-bacnet-server** as a **sibling** Compose service (BACnet **UDP 47808** + JSON-RPC on **8080**), with optional **Bearer** RPC auth via **BACNET_RPC_API_KEY** (same secret on **diy-bacnet** and **api**).
- **Caddy** (default in this repo: **HTTP-only** Caddyfile.local-dev on **127.0.0.1:18080**; on Boss Pi set **CADDY_HTTP_PORTS=18080:80** in **.env** so the UI is reachable from the LAN — see **.env.example** (avoid **0.0.0.0** in **.env** if the file has Windows CRLF; it can trigger **Invalid ip address: 0.0.0.0**). Optional TLS + Basic Auth + gateway header via **docker/caddy/Caddyfile**.
- **nginx** serves the static React build under **/app8/**.

The operator model borrows **one** idea from classic **Volttron**-style edge stacks (an optional host **vctl** hook exposed by the API for power users); **everything else here is Docker + easy-aso + diy-bacnet-server**.

2.1 1. Install Docker Engine on Raspberry Pi OS

These steps work on **64-bit Raspberry Pi OS (Bookworm)** on **Pi 5** and **Pi 3B+** (ARM). Use a **64-bit** OS image where possible.

2.1.1 1.1 Update the system

```
sudo apt update && sudo apt full-upgrade -y
sudo reboot    # optional but recommended after a large upgrade
```

2.1.2 1.2 Install Docker on Raspberry Pi OS

On newer Raspberry Pi OS releases, the Docker convenience script may point at a repo that is not published yet for your codename. If that happens, use the distro package instead.

Try the convenience script first:

```
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker "$USER"
```

If that fails with a Docker apt repo error, install the distro package path instead:

```
sudo apt update
sudo apt install -y docker.io
sudo usermod -aG docker "$USER"
sudo systemctl enable --now docker
```

Log out and back in so the **docker** group applies.

2.1.3 1.3 Install Compose support

```
docker compose version
```

If the Compose plugin is missing, try the distro package set in this order:

```
sudo apt install -y docker-compose-plugin
# or, if the plugin package is unavailable on your Pi image:
sudo apt install -y docker-compose
```

2.1.4 1.4 Quick test

```
docker run --rm hello-world
```

2.2 2. BACnet JSON-RPC (diy-bacnet-server)

easy-aso's **BACnet JSON-RPC driver** talks to **diy-bacnet-server** (or your own JSON-RPC BACnet gateway).

This stack **builds diy-bacnet** from **docker/diy-bacnet/Dockerfile**, which **shallow-clones** **diy-bacnet-server** at build time (GitHub's **default branch**). To pin a revision for reproducible OT builds, vendor a checkout or adjust that Dockerfile. The **api** service defaults to:

```
SUPERVISOR_BACNET_RPC_URL=http://diy-bacnet:8080
```

For **OT**, set a long random **BACNET_RPC_API_KEY** in **.env**. The same value is passed to **diy-bacnet** (RPC Bearer enforcement) and **api** (easy-aso's `JsonRpcBacnetClient` sends `Authorization: Bearer ...`). Leave it empty only for quick lab setups.

Ports: BACnet **UDP 47808** is published for field devices. On the **host**, diy-bacnet's container port **8080** is published as **\$ {DIY_BACNET_PORTS:-127.0.0.1:28090:8080}** (see **.env.example**) so it does not clash with other stacks; inside Compose, **api** still uses **http://diy-bacnet:8080**. For JSON-RPC from other LAN hosts (rare; increases exposure), set **DIY_BACNET_PORTS=28090:8080**.

If you run diy-bacnet **elsewhere** instead, remove or disable the **diy-bacnet** service and set **SUPERVISOR_BACNET_RPC_URL** (and the same **BACNET_RPC_API_KEY**) accordingly. **host.docker.internal** remains available on the **api** container for host-published RPC.

2.3 3. Copy this folder to the Pi

From your laptop (example):

```
scp -r vibe_code_apps_8 ben@192.168.204.12:~/bas-lite
ssh ben@192.168.204.12
cd ~/bas-lite
```

2.3.1 3.1 Windows / PowerShell (read this if you use Windows)

- **scp copies from the machine where the command runs.**
Run **scp** on your **Windows PC**, not inside **ssh** on the Pi. On the Pi there is no `C:\Users\...` path and no `$env:USERPROFILE`.
- **Do not** scp from the Pi to 192.168.204.11 unless your PC runs an **SSH server** and the firewall allows it; that path usually **times out** or **connection refused**.
- **PowerShell is not bash:** `cp -n` does not exist. Use: `if (-not (Test-Path .env)) { Copy-Item .env.example .env }`. The Pi's home folder is **~/bas-lite on Linux**, not `C:\Users\ben\bas-lite`.

Update only docker-compose.yml + .env.example from your repo clone (in PowerShell, from the `vibe_code_apps_8` directory):

```
cd $env:USERPROFILE\Documents\py-bacnet-stacks-playground\vibe_code_apps_8
.\sync-bas-lite-to-bosspi.ps1
```

By default this runs a **full deploy**: local `npm run build`, copies `frontend/dist` (Pi Docker skips Vite), enables **SD-friendly** bootstrap on the Pi, and runs **docker compose**. Use **-SyncOnly** if you only want files and will run **bootstrap-bas-lite.sh** yourself.

Or manually:

```
cd $env:USERPROFILE\Documents\py-bacnet-stacks-playground\vibe_code_apps_8
scp .\docker-compose.yml ben@192.168.204.12:~/bas-lite/
scp .\docker-compose.easy-aso-agents.example.yml ben@192.168.204.12:~/bas-lite/
scp .\env.example ben@192.168.204.12:~/bas-lite/
```

Then SSH to the Pi and edit `~/bas-lite/.env` (see §4.0 for `CADDY_HTTP_PORTS` LAN lines).

2.3.2 3.2 Scripts quick reference (Windows vs Pi)

Script	Where it runs	Purpose
<code>sync-bas-lite-to-bosspi.ps1</code>	Windows	Default: full deploy — <code>npm run build</code> , <code>scp</code> stack + <code>frontend/dist</code> , Pi <code>.env</code> gets <code>FRONTEND_SKIP_NODE_BUILD=1</code> , SSH <code>bootstrap-bas-lite.sh --sd-friendly</code> , then <code>docker compose</code> . -SyncOnly: copy files only. -SdFriendly: <code>\$false</code> / -PrebuiltFrontend: <code>\$false</code> to opt out.
<code>deploy-app8-to-bosspi.ps1</code>	Windows	Same behavior and defaults as <code>sync</code> (or use -GitDeploy for clone-on-Pi path; prebuilt UI is skipped there).
<code>run-bas-lite.ps1</code>	Windows	Local Docker Desktop only: <code>compose build / up</code> , optional -FollowLogs . Does not touch the Pi.
<code>scripts/bootstrap-bas-lite.sh</code>	Pi (or via SSH from deploy)	Merge <code>.env</code> , optional <code>--sd-friendly</code> , <code>--env-only</code> , <code>--git-update</code> , <code>--refresh-diy-bacnet</code> , <code>--diy-bacnet-tests</code> . Default <code>compose</code> path: <code>down → build → up -d</code> . Run <code>./scripts/bootstrap-bas-lite.sh --help</code> for the full flag list.

2.4 4. Build and start BAS Lite

```
cd ~/bas-lite    # folder that contains docker-compose.yml
cp .env.example .env
# Edit .env: set BACNET_RPC_API_KEY for OT. For TLS ingress, also
# set CADDYFILE, BAS_LITE_GATEWAY_TOKEN, BAS_AUTH_HASH.

docker compose build
docker compose up -d
docker compose ps
docker compose logs -f caddy
```

2.4.1 4.0a Bootstrap script (after sync-bas-lite-to-bosspi.ps1 -SyncOnly)

If you synced with **-SyncOnly** (files only), run bootstrap on the Pi:

```
cd ~/bas-lite
chmod +x scripts/bootstrap-bas-lite.sh    # once, if sync did not
# preserve +x
./scripts/bootstrap-bas-lite.sh --sd-friendly
```

A default **sync-bas-lite-to-bosspi.ps1** (no **-SyncOnly**) already runs bootstrap from the PC over SSH, with **SD-friendly** and **prebuilt UI** defaults.

scripts/bootstrap-bas-lite.sh merges **.env** from **.env.example** (if missing), appends **bosspi.env** when **CADDY_HTTP_PORTS** is not already set, replaces placeholder **BACNET_RPC_API_KEY** with a random hex secret (diy-bacnet and **api** both read **BACNET_RPC_API_KEY** — same pattern as Open-FDD generating **OFDD_BACNET_SERVER_API_KEY**), strips Windows **\r**, then runs **docker compose down**, **docker compose build**, **docker compose up -d**. Use **--env-only** to patch **.env** without Docker; **--help** for all flags (**--sd-friendly**, **--git-update**, etc.).

The **api** image installs **easy-aso[platform]==0.1.7** from **PyPI** via **pip install .** in **docker/bas_lite_api/Dockerfile** (see **docker/bas_lite_api/pyproject.toml**; **bas-lite-api** package version **1.5.0**). Bump the pin there when you intentionally adopt a newer **easy-aso** release, then rebuild **api**.

2.4.2 4.0 How you reach the UI (loopback vs LAN)

By default, **Caddy** is published on `127.0.0.1:$`
`{CADDY_HTTP_HOST_PORT:-18080}` (and HTTPS on `127.0.0.1:$`
`{CADDY_HTTPS_HOST_PORT:-18443}`) unless you override with
`CADDY_HTTP_PORTS` / `CADDY_HTTPS_PORTS`. That default means the
operator UI is **only on the Pi itself** unless you use LAN port
specs or a tunnel.

Goal	What to do
Browser on the Pi	Open <code>http://127.0.0.1:18080/app8/</code> (lab <code>Caddyfile.local-dev</code>) or <code>https://</code> <code>127.0.0.1:18443/</code> if you mapped TLS to 18443 and use <code>Caddyfile</code> .
Laptop on the same LAN (“dial in”)	In <code>.env</code> , set <code>CADDY_HTTP_PORTS=18080:80</code> and <code>CADDY_HTTPS_PORTS=18443:443</code> (two-part forms bind on all interfaces — no <code>0.0.0.0</code> text, so a Windows CRLF <code>.env</code> cannot break the IP parser). Allow the port on the Pi firewall (e.g. <code>sudo ufw allow 18080/tcp</code>). Then open <code>http://<pi-ip>:18080/app8/</code> .
Keep loopback, still use your laptop	SSH local forward: <code>ssh -L</code> <code>18080:127.0.0.1:18080 ben@<pi-ip></code> then browse <code>http://127.0.0.1:18080/app8/</code> on the laptop.

TLS + Basic Auth + gateway token (segmented OT LAN):

in `.env` set `CADDYFILE=docker/caddy/Caddyfile`,
`BAS_LITE_GATEWAY_TOKEN`, `BAS_AUTH_*`, and publish **80/443** on the
LAN, for example:

```
CADDY_HTTP_PORTS=80:80
CADDY_HTTPS_PORTS=443:443
```

Then open `https://<pi-ip>/app8/` (port **443**). Your browser will
warn about **Caddy’s internal CA** — trust it on the OT
workstation or install Caddy’s local root (see [Caddy tls internal](#)).
You will be prompted for **Basic Auth** (operator credentials
from `.env`). HTTP on port **80** redirects to HTTPS.

If you still see Invalid ip address: 0.0.0.: remove any `CADDY_HTTP_BIND=0.0.0.0` line from `.env`, run `sed -i 's/\r$//' .env` on the Pi (strip Windows carriage returns), then use `CADDY_HTTP_PORTS` as above.

2.4.3 4.1 FastAPI OpenAPI (operator debugging)

Caddy exposes `/openapi.json` for the machine-readable spec. The React UI already uses the path `/docs` for in-app BACnet notes, so **Swagger UI is not mounted on `/docs` through Caddy.**

To browse **Swagger** (`/docs` on the API container), port-forward from your laptop. If `BAS_LITE_GATEWAY_TOKEN` is set on **api**, direct calls to the container must send header `X-Bas-Lite-Gateway-Token: Bearer <same value>` (except `GET /api/v1/health`). For a simple lab, temporarily unset `BAS_LITE_GATEWAY_TOKEN` on **api** while using SSH forward only.

```
ssh -L 8090:127.0.0.1:8090 ben@PI_IP
# then open http://127.0.0.1:8090/docs
```

REST CRUD for devices/points: `/api/v1/*` (see [easy-aso supervisor docs](#)).

2.5 5. SD card wear — logs and Docker writes

Pi SD cards fail faster with **constant logging to the card.**

2.5.1 5.1 Compose logging (already set)

`docker-compose.yml` uses the `local` log driver with **size caps** for `api`, `frontend`, and `caddy`. That avoids huge JSON log files on disk.

2.5.2 5.2 Prune periodically

```
docker system prune -f
```

2.5.3 5.3 SQLite data volume

Supervisor state lives in Docker volume **bas_lite_data** mounted at **/data** in the API container (supervisor.sqlite, schedule JSON, driver config files). That is **bounded** application state — not high-churn logs.

2.6 6. Security model (OT LAN)

This is **not** internet hardening (no rate limits). It **is** meant for a **segmented OT / BAS LAN**:

Layer	What it does
Caddy tls internal	HTTPS with a self-signed site cert (trust Caddy's local CA or accept the browser warning).
Basic Auth	Operator login before the SPA or API paths load through Caddy (BAS_AUTH_USER + BAS_AUTH_HASH bcrypt in .env).
BAS_LITE_GATEWAY_TOKEN	Caddy injects X-Bas-Lite-Gateway-Token: Bearer ... on /app8/api/* , /api/v1/* , and /openapi.json only. The api container checks it when the env var is set. The browser never holds this secret. Caddy drops any client-supplied value with the same header name before proxying.
BACNET_RPC_API_KEY	diy-bacnet-server's JSON-RPC Bearer auth; easy-aso sends the same header when calling RPC.

Do not publish the **api** container port **8090** to the host in production; only **Caddy** should be reachable on **80/443**. Keep **SQLite** and driver data on the named volume.

Lab without TLS: set `CADDYFILE=docker/caddy/Caddyfile.local-dev` in `.env` and leave `BAS_LITE_GATEWAY_TOKEN` empty on **api**. That stack uses HTTP on port **80** only (no Basic Auth in Caddy — use only on an isolated lab network).

2.7 7. Related easy-aso documentation

Supervisor workflows and BACnet drivers:

SUPERVISOR_WORKFLOWS. OT-oriented notes on RPC secrets and exposure: **Supervisor OT security**.

2.8 8. Developer loop on a PC (optional)

```
cd frontend
$env:VITE_DEV_PROXY_TARGET="http://192.168.204.12"
npm install
npm run dev
```

`vite.config.ts` proxies `/app8/api` to the Pi API.

2.9 9. Runtime model

Supervisory polling, device/point config, and HTTP APIs are handled by **easy-aso** inside the **api** container. The React app keeps the same `/app8/api/*` JSON shape for pages like Overview, Live points, occupancy schedule (weekly visual + holidays + supervisor point bindings), and **System** (host metrics, Docker container table with **role** labels, **restart** / **stop** / **start**, log tail + SSE stream — including optional **easy-aso** sidecars).

Edge BACnet: only **diy-bacnet** binds **UDP 47808** on the host. Agents and the API use `JsonRpcBacnetClient` to the gateway (**device instance** addressing — see `.env.example`).

2.9.1 Optional outside-air share (easy-aso-oat, profile oat)

Legacy **optional** service **easy-aso-oat** (profile **oat**) runs a small Python loop: **read** one object via JSON-RPC, **write** to **OAT_TARGET_WRITES**. No second BACnet UDP listener.

```
docker compose --profile oat up -d easy-aso-oat
```

See `.env.example` for **OAT_SOURCE_***, **OAT_INTERVAL_SEC**, **OAT_TARGET_WRITES**.

2.9.2 Multi-agent EasyASO sidecars (profile agents)

Three **optional** services share one image built from `docker/easy_aso_agent/`: `pip install easy-aso[platform]==0.1.7`, `CMD easy-aso-agent run`, app agents on `PYTHONPATH=/app`. Each process is a `RpcDockedEasyASO` subclass (`easy_aso.runtime.rpc_docked` on PyPI); BACnet I/O is JSON-RPC to **diy-bacnet** only (`bacnet_rpm` is supported when the gateway exposes it).

Compose service	Label bas-lite.easy-aso.role	Module / class	Role
easy-aso-agent-oat	oat-share	agents.oat_share_agent / OatShareAgent	Same OAT fan-out as easy-aso-oat , full on_step lifecycle. Config-driven VAV cooling/heating request counts + optional trim-response write (<code>EASY_ASO_GL36_VAV_C</code> JSON).
easy-aso-agent-gl36-vav	gl36-vav-requests	agents.gl36_vav_requests_agent / GL36VavRequestsAgent	Generic SAT setpoint reset from zone ten (<code>EASY_ASO_GL36_AHU_C</code> JSON).

Bring them all up:

```
docker compose --profile agents up -d easy-aso-agent-oat easy-aso-agent-gl36-vav easy-aso-agent-gl36-ahu
```

Shared env pattern: **SUPERVISOR_BACNET_RPC_URL**, **SUPERVISOR_BACNET_RPC_ENTRYPOINT**, **BACNET_RPC_API_KEY**, plus per-service **EASY_ASO_AGENT_*** and step / JSON config vars (see `.env.example`).

Fourth agent (optional): merge `docker-compose.easy-aso-agents.example.yml` for `easy-aso-agent-extra` (defaults to `easy_aso.runtime.sample_agent` / `SampleAgent`).

Edit Python under `docker/easy_aso_agent/agents/` and rebuild the image (or bind-mount that folder in a local override compose file).

2.10 10. BACnet bench workflow on Boss Pi

A practical beginner loop after the stack is up:

1. Confirm containers are healthy:

```
docker compose ps
docker compose logs --tail=100 diy-bacnet api caddy frontend
# With profile agents:
# docker compose logs --tail=80 easy-aso-agent-oat easy-aso-agent-gl36-vav easy-aso-agent-gl36-ahu
```

1. Confirm the local JSON-RPC BACnet surface answers on the Pi host:

```
curl -sS "http://127.0.0.1:${DIY_BACNET_HOST_PORT:-28090}/"
```

1. Open the operator UI through Caddy:

```
http://127.0.0.1:18080/app8/
```

From another machine, either set **CADDY_HTTP_PORTS=18080:80** and use **http://<pi-ip>:18080/app8/**, or use an SSH tunnel (see §4.0). With the **TLS** Caddyfile and **CADDY_HTTP_PORTS=80:80 / CADDY_HTTPS_PORTS=443:443**, use **https://<pi-ip>/app8/**.

1. Add or verify devices and points through the easy-aso API, then confirm the legacy SPA endpoints reflect them:

```
curl -k -u operator:YOUR_PASSWORD https://<pi-ip>/app8/api/health
curl -k -u operator:YOUR_PASSWORD https://<pi-ip>/app8/api/devices
curl -k -u operator:YOUR_PASSWORD https://<pi-ip>/app8/api/points
```

1. Exercise BACnet discovery and point reads from diy-bacnet, then verify the BAS Lite pages update.

If discovery finds two bench BACnet devices, capture their addresses, learned points, and any writeable objects in your bench notes before you change anything.

2.11 11. Troubleshooting

Symptom	Check
UI loads but no BACnet values	<code>docker compose logs api</code> — RPC URL / BACNET_RPC_API_KEY mismatch? From the host: <code>curl -sS "http://127.0.0.1:\${DIY_BACNET_HOST_PORT:-28090}/"</code> should return 200 .
502 from Caddy	<code>docker compose ps</code> — is api healthy? <code>docker compose logs api</code> .
401 on API through Caddy	BAS_LITE_GATEWAY_TOKEN must match on caddy and api ; Caddy must use the default Caddyfile (not local-dev) when the token is set.
Caddy will not start — port 443 in use	Common on Windows (IIS, Hyper-V, other VPN or proxy tools). Either stop the other

Symptom	Check
listen udp4 ... 47808: bind: address already in use	listener or change the compose mapping to e.g. 8443:443 under caddy.ports and browse https://<host>:8443. Only one BACnet/IP listener per host on UDP 47808. Run sudo ss -ulnp and find 47808 — often another BACnet stack. Stop that process (or remove its container), then docker compose up -d. Optional: BACNET_UDP_HOST_PORT in .env (see .env.example) for lab-only; standard field traffic stays on 47808.
Cannot write setpoint	Driver must be bacnet_jsonrpc for that device; confirm diy-bacnet write RPC works.
Out of disk	docker system df, prune images, shrink log files. Prefer PC-built UI: deploy/sync defaults set
Frontend image build OOM on Pi	FRONTEND_SKIP_NODE_BUILD=1 and sync frontend/dist. Or raise FRONTEND_NODE_MAX_OLD_SPACE in .env for in-Docker Vite (see .env.example).
Agents never start	They are behind profile agents — run docker compose --profile agents up -d ... (see §9).

2.12 12. Stack tradeoffs

You get **asyncio**, **typed FastAPI**, **SQLite-backed** config, **hot reload** of drivers, **Docker-first** packaging, **Caddy** edge routing, and **PyPI-installable** core (easy-aso). The trade is a focused, smaller stack with fewer moving parts.

When you outgrow the shim, point the React app at `/api/v1` directly and drop legacy `/app8/api` routes.

3 Easy Aso Upstream Multi Agent Draft

4 Multi-agent runtime and RPC-docked agents — draft for easy-aso repository

Purpose: Paste or adapt this document into the **easy-aso** repo (e.g. docs/MULTI_AGENT_RUNTIME.md or an RFC issue). Demo repos such as **py-bacnet-stacks-playground** should stay **thin**: Compose services, `.env`, and **declarative agent definitions**—not duplicate runtime code.

Status in vibe_code_apps_8 (this repo): Implemented for the BAS Lite stack — **easy-aso[platform]==0.1.7**, **easy-aso-agent run**, **RpcDockedEasyASO** from PyPI, three profile-agents services (**OAT share**, **GL36 VAV requests**, **GL36 AHU SAT reset**) plus optional **docker-compose.easy-aso-agents.example.yml**. Operator runbook: docs/**BOSS_PI_BAS_LITE_DOCKER**.md §9. The sections below remain a useful upstream / library-oriented sketch.

4.1 1. Goals

Goal	Description
One BACnet edge	A single process (or dedicated gateway container) owns BACnet/IP UDP ; agents never compete for port 47808.
Many control agents	Arbitrary EasyASO subclasses run in separate processes/containers , each with <code>on_start</code> / <code>on_step</code> / <code>on_stop</code> .
PyPI is source of truth	easy-aso ships RpcDockedEasyASO , runner CLI , manifest schema , and env conventions . Demos only pin a version and point config at classes .
Optional orchestration	Docker Compose , <code>systemd</code> , or a future “agent manager” are deployments ; the library exposes a stable contract they can call.

Non-goals for **v1** in the library:

- Owning Docker or Kubernetes APIs (belongs in deployment repos or a small **agent-manager** sidecar).
- Implementing full VOLTTRON parity (VIP bus, historian, etc.).

4.2 2. Conceptual model (VOLTTRON analogy)

VOLTTRON idea	easy-aso mapping
Platform	Supervisor (api container) + BACnet gateway (e.g. diy-bacnet JSON-RPC)
Platform driver (BACnet)	Single gateway; <code>JsonRpcBacnetClient</code> as the edge driver from agents
Agent	

VOLTRON idea	easy-aso mapping
	EasyASO subclass; RpcDockedEasyASO when I/O is RPC-only
Many agents	N processes/containers , each pip install easy-aso==X and same RPC URL + auth

4.3 3. Proposed package layout (inside easy-aso)

```

easy_aso/
  easy_aso.py                                # unchanged public surface
  bacnet_client/
    jsonrpc_client.py
  runtime/
    __init__.py
    rpc_docked.py                            # RpcDockedEasyASO (moved from
demos)
  transports/
    __init__.py
    jsonrpc.py                              # optional: thin wrapper + env →
client factory
  runner.py                                  # importlib +
asyncio.run(agent.run())
  env.py                                     # parse SUPERVISOR_BACNET_RPC_* /
EASY_ASO_AGENT_*
  manifest.py                               # load/validate agent manifest
(dataclass + JSON Schema)
  heartbeat.py                              # optional HTTP POST or callback
hook (P1)
  cli/
    __init__.py
    agent.py                                # console_scripts: easy-aso-agent
= easy_aso.cli.agent:main

```

Optional (later) top-level extras in `pyproject.toml`:

- `[project.optional-dependencies] agents = []` — nothing extra, or `agents = ["jsonschema"]` if manifest validation pulls it in.
-

4.4 4. Public API (minimal v1)

4.4.1 4.1 RpcDockedEasyAS0

Module: `easy_aso.runtime.rpc_docked`

- Subclasses **EasyAS0**.
- **`async def create_application(self) -> None`**: construct **`JsonRpcBacnetClient`** from **`typed config`** (see §5); set `self.app = None`; keep `optimization_enabled_bv` placeholder behavior compatible with core **EasyAS0**.
- **`async def close_rpc_dock(self) -> None`**: idempotent close of HTTP client.
- **`bacnet_read / bacnet_write`**: delegate to `_rpc` (same signatures as **EasyAS0**).
- **`bacnet_rpm`**: default `NotImplementedError` with docstring pointing to per-object reads unless gateway adds RPM later.

Config source: Prefer **`BacnetRpcConfig`** dataclass passed explicitly in tests; production uses **`from_env()`** in `easy_aso.runtime.env`.

4.4.2 4.2 Runner

Module: `easy_aso.runtime.runner`

```
def run_agent_class(
    module: str,
    class_name: str,
    *,
    no_bacnet_server: bool = True,
) -> None:

    """Set sys.argv for EasyAS0, import module, instantiate
    class, asyncio.run(agent.run())."""
```

Environment fallbacks (if module/class_name are None):

- EASY_ASO_AGENT_MODULE
- EASY_ASO_AGENT_CLASS

4.4.3 4.3 CLI (entry point)

Console script: easy-aso-agent

```
easy-aso-agent run [--module MOD] [--class CLASS]
```

Defaults read from env; intended **Docker CMD:** easy-aso-agent run.

4.5 5. Configuration contract (env + optional manifest)

4.5.1 5.1 Environment variables (canonical names)

Shared with supervisor and demos:

Variable	Meaning
SUPERVISOR_BACNET_RPC_URL	Base URL (no trailing slash), e.g. http://diy-bacnet:8080
SUPERVISOR_BACNET_RPC_ENTRYPOINT	Path prefix, e.g. /api
BACNET_RPC_API_KEY	Optional Bearer token for outbound RPC

Agent identity (runner):

Variable	Meaning
EASY_ASO_AGENT_MODULE	Dotted import path, e.g. my_site.agents.oat_bridge
EASY_ASO_AGENT_CLASS	Class name, e.g. OatBridgeAgent

Optional tuning (agent-defined; document only as **conventions**, not required by core):

Variable	Meaning
EASY_ASO_STEP_SEC	Hint for sleep between on_step in sample agents
EASY_ASO_LOG_LEVEL	INFO, DEBUG, ...

Implementation: `easy_aso.runtime.env.load_rpc_config()` → `BacnetRpcConfig`.

4.5.2 5.2 Agent manifest (JSON or YAML) — P1 but specify now

Path: `EASY_ASO_MANIFEST_PATH` or embed in Compose labels as base64 (avoid if possible).

Purpose: Declarative metadata for UI, policy, and **future** agent-manager without Python imports.

```
# easy_aso_agent_manifest v1
schema_version: 1
agent_id: oat-share-001
display_name: "OAT share bridge"
entry:
  module: my_site.agents.oat
  class: OatAgent
runtime:
  easy_aso_version_pin: ">=0.1.5,<0.2"
bacnet:
  mode: jsonrpc_only          # future: msapi, etc.
rpc:
  url: ${SUPERVISOR_BACNET_RPC_URL}  # substitution optional
  entrypoint: ${SUPERVISOR_BACNET_RPC_ENTRYPOINT}
policies:                      # P2 – optional write scopes
  write_allow:
    - device: "10.0.1.5"
      objects: ["analog-value,2"]
```

Ship `easy_aso/runtime/schemas/agent_manifest_v1.json` (JSON Schema) and validate in `easy_aso.runtime.manifest`.

4.6 6. Logging and observability (P1)

- **Structured log line prefix:** `easy_aso.agent_id=<id>` when `EASY_ASO_AGENT_ID` set.
 - `easy_aso.runtime.heartbeat`: optional periodic POST to `EASY_ASO_HEARTBEAT_URL` with JSON `{"agent_id","status","step"}` — **off by default**; demos enable via env.
-

4.7 7. Version compatibility

- `easy_aso.__version__` checked by runner on startup (warning only in v1; hard fail optional via `EASY_ASO_STRICT_VERSION=1`).
 - **Manifest** `runtime.easy_aso_version_pin` validated with `packaging.specifiers` if present.
-

4.8 8. What demo repos keep after this lands (config-only ideal)

Artifact	Demo repo responsibility
<code>docker-compose.yml</code>	Services: <code>diy-bacnet</code> , <code>api</code> , <code>frontend</code> , minimal agent image that FROM <code>python:...</code> + <code>pip install easy-aso[platform]==X</code> + CMD <code>easy-aso-agent run</code>
<code>.env</code>	RPC URL, API key, <code>EASY_ASO_AGENT_*</code>
<code>agents/</code> or single wheel	Only site-specific subclasses (or private package); no copy of <code>RpcDockedEasyASO</code>
Docs	Link to easy-aso published docs for multi-agent

Example minimal Dockerfile (demo):

```
FROM python:3.12-slim
RUN pip install --no-cache-dir "easy-aso[platform]==0.1.6"
COPY agents/ /app/agents/
WORKDIR /app
ENV PYTHONPATH=/app
CMD ["easy-aso-agent", "run"]
```

4.9 9. Migration checklist (from current BAS Lite demo)

1. **Release** easy-aso with `easy_aso.runtime.rpc_docked`, `runner`, `cli.agent`.
 2. Bump pin in `bas_lite_api` and agent images to that release.
 3. **Delete** duplicated `rpc_docked_easy_aso.py` / shim `run_agent.py` from demo Docker contexts; replace `CMD` with `easy-aso-agent run`.
 4. Keep **Compose profiles** (`oat`, `agents`) as **pure config**.
 5. Optional: add `agent_manifest_v1` validation in a future `agent-manager` service (separate repo or demo).
-

4.10 10. Phased delivery

Phase	Scope
P0	Move RpcDockedEasyASO + runner + easy-aso-agent CLI + <code>env.load_rpc_config</code> into easy-aso ; tests; docs page Multi-agent (RPC-docked) .
P1	Manifest schema + validator; heartbeat hook; <code>EASY_ASO_AGENT_ID</code> logging.
P2	Write policy enforcement (library helper; enforcement host may be supervisor or manager).
P3	

Phase	Scope
	Optional RPM over RPC if gateway exposes a batch method (gateway-specific).

4.11 11. Test matrix (upstream)

- Unit: `RpcDockedEasyASO.create_application` mocks `JsonRpcBacnetClient`.
- Integration: mock HTTP server matching diy-bacnet JSON-RPC contract.
- CLI: `easy-aso-agent` run with `EASY_ASO_AGENT_MODULE/CLASS` pointing at a trivial subclass.

4.12 12. Appendix — Issue titles for easy-aso tracker

1. [P0] Add `easy_aso.runtime.rpc_docked.RpcDockedEasyASO`
2. [P0] Add `easy_aso.runtime.runner` + `easy-aso-agent` CLI
3. [P0] Document env vars `SUPERVISOR_BACNET_RPC_*` / `EASY_ASO_AGENT_*`
4. [P1] `agent_manifest_v1` JSON Schema + `manifest.load()`
5. [P1] Optional heartbeat URL emitter
6. [P2] Write-scope policy helper (no enforcement by default)

End of draft — copy into **easy-aso** and trim or expand as maintainers prefer.

5 Bas Lite App8 Tutorial

6 BAS Lite App 8 Tutorial

This App 8 stack runs as a **Docker Compose** deployment with:

- **caddy** — edge proxy (TLS + auth optional)
- **frontend** — static SPA under `/app8/` (nginx; build on PC by default for Pi deploys — see `deploy-app8-to-bosspi.ps1`)
- **api** — `bas_lite_api` + **easy-aso** supervisor (**PyPI easy-aso[platform]==0.1.7**), legacy `/app8/api/*` and `/api/v1/*`
- **diy-bacnet** — BACnet **UDP 47808** + JSON-RPC gateway (single field stack owner)
- **Optional profile oat** — `easy-aso-oat` lightweight OAT copy loop
- **Optional profile agents** — three `RpcDockedEasyASO` sidecars (**OAT share**, **GL36 VAV requests**, **GL36 AHU SAT reset**) via `easy-aso-agent run`

If you are setting up a new host, use `docs/BOSS_PI_BAS_LITE_DOCKER.md` as the primary operator runbook (includes Windows `sync-bas-lite-to-bosspi.ps1` / Pi `bootstrap-bas-lite.sh`).

6.1 BACnet discovery in this stack

Discovery is not performed by a dedicated `/discover` endpoint in `bas_lite_api`. Runtime BACnet operations are executed through the JSON-RPC backend (`diy-bacnet-server`) and then represented through configured devices/points in supervisor storage.

Practical workflow:

1. Bring up the stack with `docker compose up -d`.
2. Validate the BACnet backend from the host (default published port `DIY_BACNET_HOST_PORT`, e.g. `http://127.0.0.1:28090/` — see `.env.example`).
3. Add or import devices/points through the API/UI data model workflow.

4. Confirm values and health through `/app8/api/*` and `/api/v1/*`.

6.2 Project positioning

BAS Lite is intentionally compact: **Docker Compose**, the **easy-aso** supervisor + FastAPI surfaces, **diy-bacnet** JSON-RPC, and the React operator UI. Driver JSON and SQLite state ship in the **bas_lite_data** volume. **Multi-agent** supervisory loops live in **optional** Compose services under profile **agents** (three stock **RpcDockedEasyASO** agents + optional merge from **docker-compose.easy-aso-agents.example.yml**), sharing the same JSON-RPC gateway as the API — no second BACnet **UDP 47808** bind. Profile **oat** adds the legacy **easy-aso-oat** helper. Prefer config JSON (**EASY_ASO_GL36_***, **OAT_***) over forked core library code for new buildings.

7 Replicate Any Building Lan Topology

8 Recreating BAS Lite (App 8) on any HVAC OT LAN

This note is for operators and integrators who need the same App 8 BAS Lite edge stack on a different site, VLAN, or BACnet plant layout. Nothing here assumes a particular bench hostname or IP range.

8.1 1. What must stay aligned

Layer	What to configure
Network	

Layer	What to configure
	Edge host can reach field controllers on the BACnet / OT VLAN (routing, ACLs, BBMD if used).
BACnet	diy-bacnet binds UDP 47808 on the host (only one listener per host). JSON-RPC is published separately (see <code>.env.example</code>).
Driver / supervisor	Device and point definitions under BAS_LITE_DRIVER_CONFIG_DIR (SQLite + JSON) must match what easy-aso polls and what the React UI lists.
App 8 API/UI config	Alarms, notifications, schedule JSON (/data/schedule.json by default), and optional hostedScheduleName for BACnet schedule push.
Reverse proxy	Caddy upstream URL, Basic Auth secrets, TLS policy. See <code>docker/caddy/</code> .

8.2 2. Device and point identities

The runtime polls configured devices and points using the **easy-aso** supervisor configuration.

- Use **stable string identities** that match your driver JSON (often the same as the logical device folder name).
- Prefer a naming convention that scales: `campus/building/ahu_01` or `SITE_AHU01` — avoid embedding IP addresses in the identity unless that is already your standard.

8.3 3. Site model (no Python edits for a new building)

Describe equipment and points in JSON instead of editing bespoke agent code.

Option A — external file (next to the supervisor data path or absolute path):

```
{  
  "site_model_path": "site_model.json"  
}
```

Option B — inline in config: top-level keys `devices`, `points`, optional `alarm_definitions`.

Schema summary:

- **devices**: object keyed by device id; each value may include `displayName`, `kind`, `address`, BACnet `deviceId`, `pollingEnabled`, etc.
- **points**: array of objects with `pointId`, `deviceId`, `name` (driver point name), `label`, `units`, `kind` (analog | binary), `adjustable`, optional `graphicGroup`.
- **alarm_definitions**: array of rules. Supported `conditionType` values in the supervisor today: `greaterThanSetpointPlusOffset` (needs `referencePointId` and `offset`), `boolFalse`.

Copy the local site model example used by your current deployment and align names with your BACnet plant.

Optional config keys:

- **default_trend_point_id**: default trend point for API defaults.
- **bacnet_devices**: overrides the default list taken from all device keys in the site model (useful if the edge should only subscribe to a subset).

8.4 4. Occupancy schedule + optional BACnet sidecars

- **Schedule UI** stores **version 2** JSON with `schedules[]`, weekly `mon...sun` windows, holiday overrides, equipment **assignments**, optional **bacnetBindings** (rows reference **supervisor pointId** values from your driver setup), and **hostedScheduleName** for diy-bacnet `server_update_schedule`. Use **Export JSON / AI-assisted import** on the Occupancy page for bulk edits.

- **Outside-air (legacy helper):** Compose profile `oat` → `easy-aso-oat`. Reads one BACnet object, writes `OAT_TARGET_WRITES` over JSON-RPC (no second **UDP 47808**).
- **Multi-agent EasyASO:** profile `agents` → `easy-aso-agent-oat`, `easy-aso-agent-gl36-vav`, `easy-aso-agent-gl36-ahu` (`easy-aso[platform]==0.1.7`, `easy-aso-agent run`, `RpcDockedEasyASO`). Building-agnostic JSON in `.env` (`EASY_ASO_GL36_VAV_CONFIG`, `EASY_ASO_GL36_AHU_CONFIG`, `OAT_*`). Optional fourth service: merge `docker-compose.easy-aso-agents.example.yml`. Details: `docs/BOSS_PI_BAS_LITE_DOCKER.md` §9.

8.5 5. Deploy scripts from a Windows workstation

`sync-bas-lite-to-bosspi.ps1` (or `deploy-app8-to-bosspi.ps1`) — same **Pi-first defaults**: SD-friendly bootstrap, PC `npm run build` + synced `frontend/dist` (Pi skips in-Docker Vite unless you opt out). Target another host:

```
.\sync-bas-lite-to-bosspi.ps1 -Target 'user@10.0.0.50'
.\deploy-app8-to-bosspi.ps1 -SshTarget 'user@10.0.0.50'
```

-SyncOnly — copy files only; run `./scripts/bootstrap-bas-lite.sh` on the edge yourself. **-SdFriendly:\$false** / **-PrebuiltFrontend:\$false** — power-user overrides.

Pi bootstrap: `./scripts/bootstrap-bas-lite.sh` (flags in `--help`).

8.6 6. Checklist before go-live

1. From the edge shell: `docker compose ps` and `docker compose logs --tail=100 api diy-bacnet caddy frontend` (add **easy-aso-agent-*** services if profile **agents** is enabled).
2. `curl -sS http://127.0.0.1:8080/app8/api/health` (adjust port/prefix) — `counts.devices` / `points` non-zero when the model is loaded.
3. BACnet **who-is** / **read** from the same interface the driver uses (firewall and binding).

4. Journald / SD card: follow Pi logging guidance in docs/
BOSS_PI_BAS_LITE_DOCKER.md (§5).

8.7 7. Non-BACnet drivers

The UI and site model are **driver-agnostic**: anything exposed through the selected backend contract can use the same devices / points metadata. BACnet is the reference implementation in this repo; Modbus, SNMP, or custom drivers follow the same pattern as long as device ids and point names match.